fact, these problems are solved by real-world capability systems as discussed in the paper *Capability Myths Demolished* (http://srl.cs.jhu.edu/pubs/SRL2003-02.pdf) by Mark S. Miller, Ka-Ping Yee, and Jonathan Shapiro. To take one example, it is often assumed that capabilities can be used only for discretionary access control, because the creator of an object (such as a file) can share capabilities to access that file with anyone. But in a pure capability system, communications between people are also controlled by capabilities (as is the ability to create files in the first place), so if Alice creates a new file, she can share a capability to access this file with Bob only if she has a capability allowing her to communicate with Bob. Of course, there's nothing to stop Bob asking Alice in person to perform actions on the file, but that is a problem that no access control system can prevent.

A brief history of capabilities

Capability-based security was first developed in the context of operating systems in the 1970s and has been applied to programming languages and network protocols since then. The IBM System/38, which was the predecessor of the successful AS/400 (now IBM i), used capabilities for managing access to objects. In the 1990s, the E programming language (http://erights.org) combined capability-based security with object-oriented programming to create *object-capability-based security* (or *ocaps*), where capabilities are just normal object references in a memory-safe OO programming language. Object-capability–based security fits well with conventional wisdom regarding good OO design and design patterns, because both emphasize eliminating global variables and avoiding static methods that perform side-effects.

E also included a secure protocol for making method calls across a network using capabilities. This protocol has been adopted and updated by the Cap'n Proto (https://capnproto.org/rpc.html#security) framework, which provides a very efficient binary protocol for implementing APIs based on remote procedure calls. Capabilities